



TP N° 1: Classification des fleurs iris avec un modèle d'apprentissage automatique simple.

I. Contexte et objectifs

1.1. Contexte

Les données utilisées ici sont célèbres dans le domaine des data sciences. Elles ont été collectées par Edgar Anderson [1]. Ce sont les mesures en centimètres des variables suivantes : longueur du sépale (Sepal.Length), largeur du sépale (Sepal.Width), longueur du pétale (Petal.Length) et largeur du pétale (Petal.Width) pour trois espèces d'iris : setosa, versicolor et virginica.

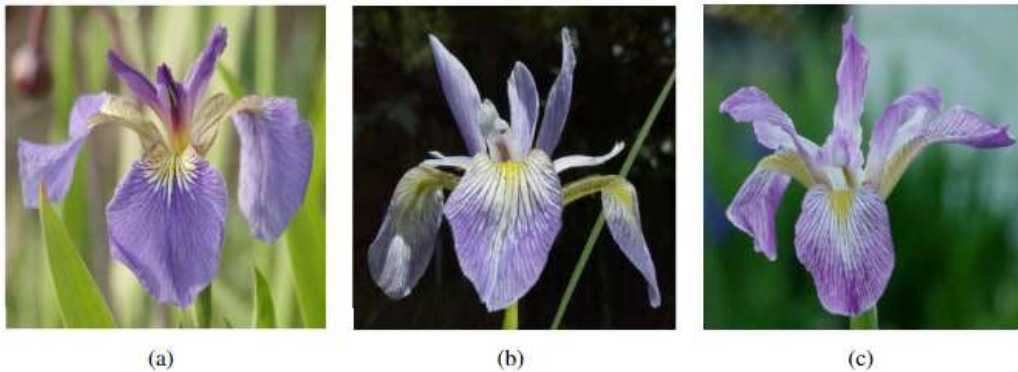


FIGURE 1 – (a) setosa (b) versicolor et (c) virginica.

Sir R.A. Fisher a utilisé ces données pour construire des combinaisons linéaires des variables permettant de séparer au mieux les trois espèces d'iris [2]. En plus de la modélisation prédictive des types de fleurs, ce TP permettra à l'étudiant de faire un petit détour sur l'analyse et la visualisation des données.

1.2. Objectifs

1. Familiariser l'étudiant avec le langage python pour la data science
2. Connaître et utiliser les bibliothèques de base de python pour la data science
3. Charger et explorer le jeu de données comme celles des Iris.
4. Préparer les données pour l'apprentissage automatique.
5. Créer et entraîner un modèle de classification simple.
6. Évaluer les performances du modèle entraîné.
7. Déployer un modèle simple pour créer une petite application.

1.3. Pré-requis

- a- Installer python et ses bibliothèques de base est nécessaires (pandas, numpy, matplotlib, seaborn, scikit-learn, etc). Si cela n'est pas déjà fait, utiliser le code suivant pour les installer.

```
!pip install pandas scikit-learn seaborn matplotlib
```

- b- Avoir suivi la formation complète de python pour l'analyse des données.



II. Les étapes de réalisation du TP

Étape 1 : Lancement de l'éditeur et Importation des bibliothèques

Si la connexion internet le permet, il est plus facile d'utiliser les éditeurs de codes python en ligne qui offrent un grand nombre d'avantages. L'un des plus utilisé actuellement est [Google Colab](#) qui offre l'avantage de collaborer sur un code partagé en plus des ressources allouées. Mais, il en existe beaucoup d'autres éditeurs en ligne comme, Deepnote, Kaggle, etc. En cas de difficultés avec les éditeurs en ligne, il est mieux d'utiliser un éditeur en local comme Spyder, Jupiter, Pycharm etc. Ces éditeurs sont accessibles à travers l'installation d'Anaconda qui est une distribution libre et open source des langages python et R.

NB : La structure du code ne changera pas selon qu'on utilise un éditeur en ligne ou en local ou encore selon le choix de l'éditeur.

- 1- Lancer l'éditeur google colab ou jupyter d'anaconda
- 2- Importer les bibliothèques de base (pandas, numpy, matplotlib, seaborn) en utilisant « import + nom de la bibliothèque » de la manière suivante.

```
Copier le code
# Importation des bibliothèques de base nécessaires
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

Étape 2 : Charger et explorer les données

- 1- Charger le fichier iris.csv dans un DataFrame en utilisant la fonction `read_csv` comme dans l'exemple suivant : `df = pd.read_csv('iris.csv')`
- 2- Afficher les premières lignes pour comprendre la structure des données.

```
# Afficher les premières lignes du jeu de données
print(df.head())
```

```
# Statistiques descriptives pour comprendre la distribution des
caractéristiques
print(df.describe())
```

1. Visualiser la répartition des espèces.

```
# Visualisation de la répartition des classes
sns.countplot(x='species', data=df)
plt.title('Distribution des espèces d\'iris')
plt.show()
```

Exercices : Visualisation des données d'iris

Exercice 1 :



La dernière colonne du tableau des données iris contient le nom des espèces (species) réparties en trois catégories: *setosa*, *versicolor* et *virginica*. Pour accéder à celle-ci, il faut utiliser les agrégation d'excel. On dit que la dernière colonne contient une variable **qualitative** à trois modalités.

- 1- Afficher l'effectif de chacune des 3 modalités.
- 2- Tracer les diagrammes de ces effectifs sous forme de:
 - a- histogramme;
 - b- Secteurs;
 - c- Barres groupées;
 - d- Cascade;
- ...
- 3- choisir les meilleures colorations et étiquetage.
- 4- Quelle est la meilleure représentation visuelle ?

Exercice 2 :

La troisième colonne (*Petal.Length*) du jeu de données iris contient la longueur du pétale. Il s'agit d'une variable mesurée qualifiée alors de variable quantitative.

- 1- Résumer l'information contenue dans cette variable.
- 2- Faire une visualisation adéquate de cette variable par un histogramme.
- 3- Réaliser le même type d'analyse sur chacune des autres variables quantitatives: largeur du pétale (**Petal.Width**), longueur du sépale (**Sepal.Length**) et largeur du sépale (**Sepal.Width**).

Exercice 3 :

Une fois réalisés les graphiques pour chaque variable prise séparément, l'étude peut porter sur la relation entre deux variables. On parle de croisement de deux variables ou d'étude bivariée. La représentation graphique liant deux variables quantitatives est le nuage de points.

1. Représenter par exemple la longueur et la largeur du pétale pour les 150 iris contenus dans le fichier de données. Commentaire.
2. Réaliser l'étude du croisement de deux variables quantitatives de votre choix. Il est clair que le sens biologique de l'étude ne doit pas être négligé.

Exercice 4 :

La représentation graphique permettant de lier une variable qualitative et une variable quantitative est *la boîte à moustaches* (**BOXPLOT**).

- 1- Représentons par exemple la longueur des pétales en fonction de l'espèce (Species). Commenter.
- 2- Choisissez une autre variable quantitative, croisez-la avec la variable espèce (Species) d'iris et commentez.

Exercice 5 :

Le nuage de points comme les boîtes à moustaches montrent que les données morphologiques des iris semblent liées à l'espèce. Il pourrait donc être intéressant de réaliser des graphiques différents pour chacune des modalités *setosa*, *versicolor* et *virginica* ou de superposer l'information espèce (Species) dans le graphique des nuages de points.

1. Proposer quelques développements (des représentations possibles pour mettre cela en évidence).
2. Calculer les différentes corrélations et visualiser.
3. Proposer d'autres représentations possibles.

Étape 3 : Préparer les données pour le modèle



1. Séparer les données en **caractéristiques** (X) et **cible** (y).

Pour ce faire, il faut d'abord importer la bibliothèque nécessaire `train_test_split` à travers le code suivant :

```
from sklearn.model_selection import train_test_split

# Séparer les caractéristiques et la cible
X = df.drop('species', axis=1)
y = df['species']
```

2. Diviser le jeu de données en ensembles d'entraînement et de test (80%/20%).

```
# Diviser les données en ensemble d'entraînement et de test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

3. Normaliser les caractéristiques pour améliorer la performance du modèle en utilisant la bibliothèque suivante `StandardScaler`.

```
from sklearn.preprocessing import StandardScaler

# Normaliser les caractéristiques
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Étape 4 : Créer et entraîner un modèle de classification (K-Nearest Neighbors)

1. Choisir le modèle K-Nearest Neighbors (KNN) pour la classification.

```
from sklearn.neighbors import KNeighborsClassifier

# Créer le modèle KNN
knn = KNeighborsClassifier(n_neighbors=3)
```

2. Entraîner le modèle sur l'ensemble d'entraînement.

```
# Entraîner le modèle
knn.fit(X_train, y_train)
```

Étape 5 : Évaluer le modèle

1. Prédire les classes sur l'ensemble de test.

```
# Prédire les classes de l'ensemble de test
y_pred = knn.predict(X_test)
```

2. Afficher la matrice de confusion et la visualiser



```
# Afficher la matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)

sns.heatmap(conf_matrix, annot=True, cmap='Blues', fmt='d',
            xticklabels=df['species'].unique(), yticklabels=df['species'].unique())
plt.title('Matrice de confusion')
plt.xlabel('Prédictions')
plt.ylabel('Vraies classes')
plt.show()
```

3. Calculer et afficher l'exactitude, le rapport de classification et la matrice de confusion.

```
# Calculer l'exactitude
accuracy = accuracy_score(y_test, y_pred)
print(f"Exactitude du modèle : {accuracy * 100:.2f}%")

# Afficher le rapport de classification
print("Rapport de classification :\n", classification_report(y_test,
y_pred))
```

Étape 6 : Interprétation des résultats

1. Analysez les résultats du modèle, en particulier les erreurs dans la matrice de confusion.
2. Discutez de la façon dont la normalisation des données a pu influencer les performances.

Étape 7 : Optimisation du modèle et comparaison

1- Optimisation des hyper-paramètres

- a- La première étape du projet consiste à choisir les hyper paramètres qui influencent les performances des modèles. Pour le cas du modèle KNN, il s'agira principalement de : le nombre de voisins k (1, 2, 3, 4, 5, ...) et la distance (euclidienne, Manhattan, Cosine, Minkowski, etc.).
- b- La seconde étape consiste à évaluer pour chaque configuration et choisir celle offrant la meilleure performance en utilisant la validation croisée ou validation simple. Deux approches de recherches sont utilisées (recherche en grille et recherche aléatoire). E

2- Entraînement des autres modèles et Comparaison

Essayer d'autres modèles, comme la régression logistique (Logistic Regression LR) arbres de décision (Decision Tree DT), Naive Bayes (NB), Support Vector Machine (SVM), Artificial Neural Network (ANN) et comparer leurs performances avec le modèle KNN.

Étape 7 : Déploiement du modèle et création d'une petite interface d'API

1- Le déploiement du modèle sur flask se fait en utilisant les étapes suivantes :

- a- Enregistrez le modèle en utilisant un format de sérialisation (la fonction Pickle est régulièrement utilisée) pour le charger plus tard dans Flask.



b- Déploiement du Modèle avec Flask :

- Créez une application Flask qui expose une API.
- Implémentez une route /predict qui accepte une requête POST contenant les caractéristiques nécessaires et renvoie les prédictions du modèle.
- Testez l'API localement en envoyant une requête POST avec des données de test et assurez-vous qu'elle renvoie les résultats attendus.

2- Création du Dashboard avec Streamlit :

- Créez un tableau de bord avec Streamlit pour visualiser les prédictions et l'analyse de données.
- Utilisez un formulaire dans Streamlit pour saisir les données d'entrée et envoyer une requête POST à l'API Flask pour obtenir la prédiction.
- Affichez les prédictions obtenues de l'API et ajoutez des visualisations pertinentes pour les données (histogrammes, diagrammes de dispersion, etc.).
- Ajoutez des filtres interactifs pour permettre à l'utilisateur de modifier les paramètres d'entrée et observer l'évolution des prédictions.

3- Lancement et test de l'application :

- Exécuter Flask pour démarrer l'API en local.
- Ouvrir le tableau de bord Streamlit et assurez-vous qu'il communique correctement avec l'API Flask et affiche les résultats.

Etape 8 : Conclusion du TP

Ce TP permet aux étudiants d'atteindre les objectifs fixés en introduction. Les étudiants feront tout de même part de difficultés rencontrées et n'hésiteront pas à faire des approfondissements.